Programación Procedural - Evaluación Práctica Operativa 2 – Tema 1

Duración: 70 minutos. La actividad se inicia a la hora 12:30.
1) Resuelva y codifique en Lenguaje C el enunciado presentado.
2) Terminado el programa lo guardará con su nombre, ej. “Juan Perez.cpp”
3) Compilar y ejecutar el programa con el lote de prueba dado.
4) Teniendo a la vista la salida en pantalla de su programa, tomarle una foto colocando su DNI en el extremo inferior derecho de la pantalla.
5) En el aula virtual, en la pestaña Evaluaciones, encontrará una tarea con el nombre de su grupo, donde subirá el archivo .cpp y la fotografía de la pantalla. Podrá subir su trabajo hasta la hora 13:50

Actividad:
El Centro de Salud Provincial requiere implementar un módulo de software en **Lenguaje C estándar** para gestionar el registro diario de atenciones en el servicio de guardia de emergencias. Dado que la llegada de pacientes durante el turno es impredecible y continua, los registros deben organizarse dinámicamente en el Montículo (Heap) utilizando una **Lista Enlazada Simple de Registros**. De cada paciente ingresado en guardia se conoce: Número de Historia Clínica: Entero positivo (ej. 8801, 8805...), Código de Urgencia: Carácter (‘C’: Crítica, ‘M’: Moderada, ‘L’: Leve) y Tiempo de Espera en Guardia: Número entero positivo expresado en minutos.

Requerimientos Operacionales e Ítems a Evaluar

1. **Modularización y Gestión de Memoria**: La función main() debe actuar exclusivamente como coordinadora del flujo del programa. Todos los ítems deben resolverse mediante subprogramas independientes declarados previamente con sus prototipos. Al finalizar la ejecución, el main() debe garantizar la liberación total de la memoria dinámica de los nodos de la lista enlazada utilizando la función free() para prevenir fugas de memoria (Memory Leaks).
2. **Ítem 1**: Diseñar e implementar una función independiente para realizar la carga de los pacientes por teclado e insertarlos en la lista enlazada simple alojada en el Montículo (Heap). Cada nodo debe solicitar memoria dinámicamente con malloc(). La carga finaliza cuando el usuario ingresa un Número de Historia Clínica igual a cero (0).
3. **Ítem 2**: Diseñar una función recursiva que recorra la lista enlazada y devuelva al programa principal (main) tres métricas estadísticas sobre las atenciones de la guardia, utilizando tres mecanismos distintos de transferencia de datos:
    - **Métrica A** (Retorno explícito por return): Cantidad de pacientes pertenecientes a la categoría de Urgencia Leve ('L').
    - **Métrica B** (Pasaje por dirección): El tiempo de espera Mínimo registrado entre todos los pacientes de la lista.
    - **Métrica C** (Pasaje por referencia): Cantidad de pacientes que registraron una Demora Prolongada (tiempo de espera mayor a 60 minutos).
4. **Ítem 3**: Diseñar una función recursiva que reciba un Número de Historia Clínica ingresado por teclado y busque su presencia en la lista enlazada. Si el paciente se encuentra registrado, retornar al main() el Tiempo de Espera en minutos e indicar además si su nivel de Urgencia es Crítico ('C') devolviendo un indicador (1 si es 'C', 0 en caso contrario) para ser informados en pantalla; de lo contrario, retornar un valor centinela (ej. -1) para indicar su ausencia.

Lote de Prueba Sugerido para Ejecución
Cargar la lista enlazada con los siguientes datos de pacientes de la guardia:

Historia Clínica	Código de Urgencia	Tiempo de Espera
8803	L	70 minutos
8810	M	25 minutos
8818	L	35 minutos
8825	C	10 minutos

Ítem 3: 8820 (Historia Clínica a buscar)